

(a) Identify the next two actions using UCB action selection. Assume $c = 0.7$, [2.0 M]

- UCB (Upper Confidence Bound) formula: $UCB_i = \bar{r}_i + c\sqrt{\frac{\ln t}{n_i}}$, where $\bar{r}_i$ is the average reward, t is the total number of selections, n_i is the number of times action i was selected, and c is the exploration parameter (0.7 here).
- Total time steps = 8, rewards = [30, 50, 60, 50, 50, 50, 20, 50].
- Action counts: Influencer 2 (2 times), 3 (2 times), 4 (1 time), 5 (3 times).
- Average rewards: 2 (25), 3 (15), 4 (50), 5 (16.67).
- At $t=8$, $\ln 8 \approx 2.079$, $c\sqrt{\frac{\ln t}{n_i}} = 0.7\sqrt{\frac{2.079}{n_i}}$.
 - For 2: $0.7\sqrt{\frac{2.079}{2}} \approx 0.7 \times 1.02 \approx 0.714$, $UCB = 25 + 0.714 = 25.714$.
 - For 3: $0.7\sqrt{\frac{2.079}{2}} \approx 0.714$, $UCB = 15 + 0.714 = 15.714$.
 - For 4: $0.7\sqrt{\frac{2.079}{1}} \approx 0.7 \times 1.44 \approx 1.008$, $UCB = 50 + 1.008 = 51.008$.
 - For 5: $0.7\sqrt{\frac{2.079}{3}} \approx 0.7 \times 0.83 \approx 0.581$, $UCB = 16.67 + 0.581 = 17.251$.
- Next action: Influencer 4 (highest $UCB = 51.008$). Update $n_4 = 2$, recalculate:
 - For 4: $0.7\sqrt{\frac{2.079}{2}} \approx 0.714$, $UCB = 50 + 0.714 = 50.714$.
 - Next: Influencer 4 again ($50.714 > \text{others}$).
- Answer: Influencer 4, Influencer 4.

(b) How would you choose the next two actions using ϵ -greedy? Explain. Assume $\epsilon = 0.7$, [2.0 M]

- ϵ -greedy: With probability ϵ (0.7), choose a random action; with probability $1-\epsilon$ (0.3), choose the action with the highest average reward.
- Average rewards: 2 (25), 3 (15), 4 (50), 5 (16.67). Highest = 4 ($\frac{\sum r_i}{n_i}$ 50).
- First action: 70% chance random, 30% chance Influencer 4.
- Second action: Repeat the process.
- Due to randomness, specific actions depend on the outcome, but typically, with $\epsilon=0.7$, random selection dominates. However, based on averages, Influencer 4 is likely when greedy is applied.
- **Answer: Likely Influencer 4, then random (e.g., Influencer 2 or 5), depending on random draw.**

(c) Assume the reward distribution varies with time as it is valid for such a problem. Re-estimate the value of Influencers with IDs 3 and 5. [Note: assume $\alpha = 0.5$, if required]

- Since reward distribution varies with time, use an exponential moving average: $V_t = \alpha \cdot r_t + (1 - \alpha) \cdot V_{t-1}$, where $\alpha = 0.5$, r_t is the current reward, and V_{t-1} is the previous value (initially 0).
- For Influencer 3: Rewards at steps 1 and 6 = [30, 20].
 - Step 1: $V_1 = 0.5 \cdot 30 + 0.5 \cdot 0 = 15$.
 - Step 6: $V_6 = 0.5 \cdot 20 + 0.5 \cdot 15 = 10 + 7.5 = 17.5$.
- For Influencer 5: Rewards at steps 4, 5, 7 = [50, 50, 20].
 - Step 4: $V_4 = 0.5 \cdot 50 + 0.5 \cdot 0 = 25$.
 - Step 5: $V_5 = 0.5 \cdot 50 + 0.5 \cdot 25 = 25 + 12.5 = 37.5$.
 - Step 7: $V_7 = 0.5 \cdot 20 + 0.5 \cdot 37.5 = 10 + 18.75 = 28.75$.
- **Answer: Influencer 3 = 17.5, Influencer 5 = 28.75.**

(i) Suggest any one valid set of values for $(\alpha, \beta, \gamma, \delta)$ w.r.t each of the below cases and justify your choice in no more than one sentence: [2.0 M]

- (a) If the robot works in a deterministic environment
 - Values: $(\alpha = 1, \beta = 1, \gamma = 1, \delta = 1)$. Justification: In a deterministic environment, all actions (moveEast, moveWest, moveNorth) and exit lead to certain outcomes, so probabilities can be set to 1.
- (b) If the robot's action outcomes are stochastic in nature
 - Values: $(\alpha = 0.8, \beta = 0.7, \gamma = 0.9, \delta = 0.6)$. Justification: Stochastic outcomes require probabilities less than 1 to reflect uncertainty, with values chosen to favor successful transitions based on likely paths.

(ii) Write down the Bellman expressions for $v_x(w)$, $q_x(x, \text{moveEast})$ that suit the given dynamics, i.e., make the Bellman equations to be very specific to this case [2.0 M]

- $v_x(w) = \max[q_x(w, \text{moveEast}), q_x(w, \text{moveWest}), q_x(w, \text{moveNorth}), q_x(w, \text{exit})]$
- $q_x(x, \text{moveEast}) = \sum_{s'} P(s'|x, \text{moveEast})[r(x, \text{moveEast}, s') + \lambda v_{s'}(s')] = \alpha[\lambda + v_y(y)] + (1 - \alpha)[\lambda + v_x(x)]$

(iii) Illustrate the significance of discounting using the above part ii) answer for any one of the expressions. Explain in no more than two sentences and be specific to the given use case. [1.0 M]

- Using $q_x(x, \text{moveEast})$, discounting via λ reduces the future value $v_y(y)$ or $v_x(x)$ to prioritize immediate rewards (e.g., +10 at y) over distant ones, encouraging the robot to seek the treasure quickly rather than exploring indefinitely.

(iv) Comment on the robot's behaviour if λ is zero [1.0 M]

- If $\lambda = 0$, the robot will only consider immediate rewards, likely exiting at x for +1 or avoiding the -1 at z, ignoring the +10 treasure at y due to no value placed on future states.

i. State Space: List and explain the states the robot can be in. [1.0 M]

- States: (0,0), (0,1), (0,2), (1,0), (1,1), (1,2), (2,0), (2,1), (2,2). Explanation: These represent all possible positions in the 3x3 grid where the robot can be located.

ii. Action Space: List the actions available to the robot in each state. [1.0 M]

- Actions: up, down, left, right. Explanation: The robot can move in any of these four directions from any state, subject to grid boundaries.

iii. Reward Function: Define the rewards for each state-action pair. [1.0 M]

- Rewards: -1 for each move (up, down, left, right) from any state except (2,2); +10 for reaching (2,2) and choosing any action, 0 for all actions after reaching (2,2). Explanation: The robot incurs a -1 cost per move, a +10 reward for reaching the target (2,2), and no further reward after the episode ends.

iv. State-Transition Probabilities: Describe the transition probabilities for the robot moving in the grid, assuming no stochasticity (i.e., deterministic transitions). [1.0 M]

- Transition Probabilities: 1.0 for the intended move (up, down, left, right) if within grid bounds, 0.0 otherwise (e.g., from (0,0) moving left or down results in no transition, staying at (0,0)). Explanation: Deterministic transitions mean the robot moves to the exact intended state if the action is valid, otherwise it remains in the current state.

(a) Describe dynamics in Markov Decision Process (MDP) using a neat table for this scenario, considering the states (low, medium, high, and top), actions (low-power and high-power modes), transition probabilities, and energy costs. [2.0 M]

State (s)	Action (a)	Next State (s')	$P(s' s, a)$	Reward $r(s, a, s')$	Energy Cost
low	low-power	medium	0.8	-1	1
low	low-power	low	0.2	-1	1
low	high-power	medium	0.6	-2	2
low	high-power	low	0.4	-2	2
medium	low-power	high	0.8	-1	1
medium	low-power	medium	0.2	-1	1
medium	high-power	high	0.6	-2	2
medium	high-power	medium	0.4	-2	2
high	low-power	top	0.8	-1	1
high	low-power	high	0.2	-1	1
high	high-power	top	0.6	-2	2
high	high-power	high	0.4	-2	2
top	low-power	top	1.0	0	1
top	high-power	top	1.0	0	2

- **Assumption:** Reward is the negative energy cost (-1 for low-power, -2 for high-power) as the goal is to minimize energy consumption, with 0 reward at top since the task is complete.

(b) Perform one iteration of value iteration [using asynchronous update] using a discount factor of 0.6, starting with all state values initialized to 0. Calculate the updated values for all states and determine whether the results suggest any changes to the drone's optimal policy. [3.0 M]

- **Bellman Update:** $V(s) = \max_a \sum_{s'} P(s' | s, a) [r(s, a, s') + \gamma V(s')]$, where $\gamma = 0.6$, initial $V(s) = 0$.
- **Iteration 1:**
 - **low (low-power):** $0.8[-1 + 0.6 \cdot 0] + 0.2[-1 + 0.6 \cdot 0] = 0.8 \cdot -1 + 0.2 \cdot -1 = -0.8 - 0.2 = -1$
 - **low (high-power):** $0.6[-2 + 0.6 \cdot 0] + 0.4[-2 + 0.6 \cdot 0] = 0.6 \cdot -2 + 0.4 \cdot -2 = -1.2 - 0.8 = -2$
 - $V(low) = \max(-1, -2) = -1 \rightarrow$ Use low-power.
 - **medium (low-power):** $0.8[-1 + 0.6 \cdot 0] + 0.2[-1 + 0.6 \cdot 0] = -1$
 - **medium (high-power):** $0.6[-2 + 0.6 \cdot 0] + 0.4[-2 + 0.6 \cdot 0] = -2$
 - $V(medium) = \max(-1, -2) = -1 \rightarrow$ Use low-power.
 - **high (low-power):** $0.8[-1 + 0.6 \cdot 0] + 0.2[-1 + 0.6 \cdot 0] = -1$
 - **high (high-power):** $0.6[-2 + 0.6 \cdot 0] + 0.4[-2 + 0.6 \cdot 0] = -2$
 - $V(high) = \max(-1, -2) = -1 \rightarrow$ Use low-power.
 - **top:** $1.0[0 + 0.6 \cdot 0] = 0$ (for both actions).
 - $V(top) = 0$.
- **Updated Values:** $V(low) = -1, V(medium) = -1, V(high) = -1, V(top) = 0$.
- **Policy Suggestion:** The initial policy favors low-power mode at all states to minimize energy cost, which aligns with the goal. No changes are suggested yet, as further iterations are needed to refine values, but the trend indicates low-power is optimal given the high failure rate of high-power mode.

(i) Estimate the value of the state-value (S_2 , Down) and (S_2 , Up) using first visit estimate. Show all the steps. Assume the discount factor to be 0.9. [3 Marks]

- **Episode:** S1 - Up (+2) - S2 - Down (+4) - S1 - Down (+4) - S2 - Down (+8) - S1 - Down (+8) - S2 - Down (+8) - S1 - Up (+8) - S2 - Down (+8)
- **First Visit Estimate:** Only the first visit to a state-action pair in an episode contributes to the value estimate. The return G_t is the discounted sum of rewards from the first visit onward.
- **Discount Factor (γ):** 0.9
- **Trace First Visits:**
 - S2, Down: First visit at step 3 (S2 - Down (+4)).
 - S2, Up: First visit at step 9 (S1 - Up (+8) - S2, no action reward yet, but we need the next reward).
- **Calculate Returns:**
 - For (S2, Down) at step 3:
 - Rewards after first visit: +4 (step 3), +4 (step 5), +8 (step 7), +8 (step 9), +8 (step 11).
 - Discounted return $G_3 = 4 + 0.9 \cdot 4 + 0.9^2 \cdot 8 + 0.9^3 \cdot 8 + 0.9^4 \cdot 8$
 - $G_3 = 4 + 3.6 + 6.48 + 5.832 + 5.2488 = 25.1608$
 - Value estimate for (S2, Down) = 25.1608 (single episode, so average is the return).
 - For (S2, Up) at step 9:
 - Rewards after first visit: +8 (step 11).
 - Discounted return $G_9 = 8$
 - Value estimate for (S2, Up) = 8 (single episode).
- **Final Values:**
 - $V(S2, Down) \approx 25.16$
 - $V(S2, Up) = 8$

(ii) Assume the target policy π for S2 is updated after the estimation in (a). Show the target policy for S2. [1 Mark]

- **Target Policy Update:** The target policy π is deterministic and chooses the action with the highest estimated value.
- From (i), $V(S2, Down) = 25.16$ and $V(S2, Up) = 8$, so the optimal action is Down.
- **Updated Target Policy for S2:** $\pi(S2) = Down$ (probability 1 for Down, 0 for Up).

(a) Write two issues when the behaviour policy used in Off-policy Monte Carlo learning is deterministic. [1 Marks]

- **Limited exploration:** A deterministic policy always selects the same action, reducing the variety of states visited and potentially missing optimal policies.
- **Bias in estimation:** It may fail to adequately sample less likely but potentially rewarding state-action pairs, leading to inaccurate value estimates.

(b) In the Monte Carlo method, the learning of values/policy is done using the episodes. How do we ensure the episodes generated for learning policies encourage exploration? Give one point in support of On-Policy Learning and one point in support of Off-Policy Learning. [1 Marks]

- **Ensuring Exploration:** Episodes can be generated using a stochastic behavior policy (e.g., ϵ -greedy) to introduce randomness, ensuring diverse state-action pairs are explored.
- **On-Policy Learning Support:** It directly learns the policy being used to generate episodes, ensuring the policy improves with its own experience.
- **Off-Policy Learning Support:** It allows learning from a separate behavior policy that can be designed to maximize exploration, improving the robustness of the learned policy.

(c) Write down either policy iteration or value iteration algorithm. How do you describe its complexity? Write two comments about the learned policy. [3 Marks]

- **Value Iteration Algorithm:**

- ¹ Initialize $V(s) = 0$ for all states s .

- ² Repeat until convergence:

- For each state s , update $V(s) = \max_a \sum_{s'} P(s'|s, a)[r(s, a, s') + \gamma V(s')]$.

- ³ Extract policy $\pi(s) = \arg \max_a \sum_{s'} P(s'|s, a)[r(s, a, s') + \gamma V(s')]$.

- **Complexity:** The complexity is $O(S^2 A)$ per iteration, where S is the number of states and A is the number of actions, due to evaluating all state-action pairs; the total number of iterations depends on convergence rate.

- **Comments on Learned Policy:**

- The policy is optimal given sufficient iterations, as it maximizes the expected cumulative reward.

- It may be computationally expensive to compute for large state spaces, potentially requiring approximations.